



ENTERPRISE SYSTEM ARCHITECTURE CASE STUDY

High-Throughput Event-Driven Microservices on Google Kubernetes Engine

A production-grade, distributed e-commerce application engineered with Spring Boot 3, React 19, Apache Kafka, and PostgreSQL, deployed to Google Kubernetes Engine (GKE Autopilot) with complete CI/CD and observability.

5

SPRING MICROSERVICES

2

REACT 19 FRONTENDS

4

ISOLATED POSTGRESQL
DBS

100%

EVENT-DRIVEN (KAFKA)

Technology Stack & Architecture Matrix

Backend Services

Java 21 Spring Boot 3.2

Spring Cloud Gateway

Hibernate / JPA

Flyway DB Migrations HikariCP Pool

Resilience4j

Frontend Applications

React 19 TypeScript

Vite Build System

Tailwind & Vanilla CSS

Lucide Vector Icons

Nginx Web Server

Dynamic Reverse Proxy

Cloud & Infrastructure

GKE Autopilot Docker Multi-Stage

Apache Kafka 3.7 (KRaft)

PostgreSQL 16 StatefulSet

Google Artifact Registry

Cloud Load Balancing

Cloud Storage (GCS)

Core Business & Technical Highlights

Dual Portals (Customer & Admin)

Independent, decoupled React 19 SPAs: a high-conversion storefront with product filtering, dynamic cart, and checkout, paired with a real-time admin management portal.

Transactional Outbox Pattern

Eliminates dual-write bugs during order placement. Database updates and outbound Kafka events are committed atomically, guaranteeing at-least-once event delivery.

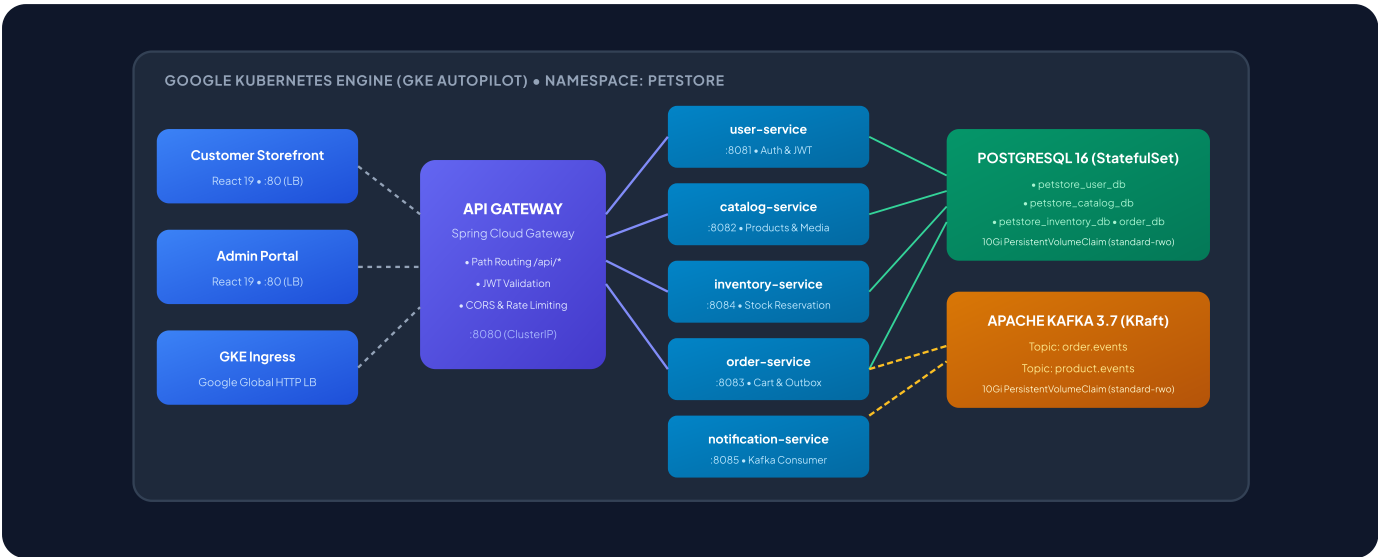
Asynchronous Event Sourcing

Order events trigger non-blocking stock reservations in Inventory Service and real-time email/push dispatches in Notification Service via Apache Kafka KRaft cluster.

Defense-In-Depth Security

Stateless JWT authentication, BCrypt password hashing, Spring Cloud Gateway centralized authorization, and zero trust internal Kubernetes network policies.





Microservices Responsibility & Schema Matrix

SERVICE NAME	PORT	DEDICATED DATABASE	KEY FUNCTIONALITY	ASYNC INTEGRATION
user-service	:8081	petstore_user_db	Authentication, JWT tokens, profile management, addresses	Synchronous REST via Gateway
catalog-service	:8082	petstore_catalog_db	35+ products, categories, dog/cat filtering, GCS media links	Publishes product inventory updates
inventory-service	:8084	petstore_inventory_db	Stock reservation, real-time availability checking, lock handling	Consumes <code>order.events</code> for stock decrement
order-service	:8083	petstore_order_db	Carts, checkout, Transactional Outbox, order status lifecycle	Publishes <code>order.events</code> via Outbox
notification-service	:8085	Stateless	Customer order confirmation alerts & status tracking notifications	Consumes <code>order.events</code> in real-time



Database-per-Service Principle: Each microservice owns its schema completely with strict network and user-level isolation in PostgreSQL. No service can directly query another service's database, preventing tight coupling and enabling independent database schema evolutions via Flyway migrations.



Multi-Cloud & Local Deployment Comparison

DEPLOYMENT STRATEGY	TARGET ENVIRONMENT	DATA LAYER	STRENGTHS & PRIMARY USE CASE
Local Docker Compose	Developer Workstation	Containerized PostgreSQL & Kafka	Instant feedback loop, zero cloud billing, rapid unit/integration testing
Strategy A (Serverless)	Google Cloud Run (8 Services)	Cloud SQL PostgreSQL 16 + Kafka VM	Pay-per-request billing, automatic scale-to-zero during non-business hours
Strategy B (GKE Autopilot)	Google Kubernetes Engine (Autopilot)	In-Cluster StatefulSets + 10Gi PVCs	Enterprise orchestration, internal DNS resolution, zero managed VM overhead

Production Kubernetes Manifest Architecture

StatefulSets with Dynamic Storage

PostgreSQL 16 and Kafka 3.7 are deployed as Kubernetes `StatefulSets` with dedicated `volumeClaimTemplates` requesting `standard-rwo` persistent disks, ensuring data survivability across pod restarts or node rescheduling.

GKE Health Checks & BackendConfig

Spring Boot Actuator health endpoints (`/actuator/health`) are integrated with Google Cloud Load Balancer via custom `BackendConfig` CRDs, ensuring zero downtime rolling updates and proactive traffic redirection.

Secret & Config Decoupling

All sensitive credentials (database passwords, JWT secret keys) are managed through Kubernetes Secrets and injected into container environments via `secretKeyRef`, preventing secrets leakage into source control.

Automated One-Click Deployment

Includes `deploy-all.sh` that orchestrates the rollout sequence: Namespace → Secrets → ConfigMaps → StatefulSets (Storage & Message Broker) → Microservices → Gateways & Ingress.

Clean Repository Structure & Git Branching Strategy

Professional Git Flow Architecture: The codebase maintains a clean, production-ready branching model where `main` represents stable releases, `develop` aggregates sprint deliverables, and localized features branch into `feature/*`. All deployment manifests are centralized in `deploy/k8s/` and `deploy/cloud-run/` without altering microservice business code.

Local Hybrid Developer Velocity

Developers run backing infrastructure (`postgres`, `kafka`) via Docker Compose while running their microservice or React frontend directly on their workstation for instant Vite HMR and sub-second Java code debugging.

Multi-Architecture Container Builds

Docker container images are built specifically with `--platform linux/amd64` multi-stage pipelines to ensure seamless binary execution between developer Apple Silicon (ARM64) Macs and Google Cloud x86_64 nodes.



Automated End-to-End Validation Results

VERIFICATION STEP	TARGET COMPONENT	OBSERVED LATENCY	OUTCOME
1. Product Catalog Query	GET /api/products	~42 ms	PASSED (35 PRODUCTS)
2. Customer Registration	POST /api/auth/register	~68 ms	PASSED (JWT GENERATED)
3. Shipping Address Creation	POST /api/addresses	~35 ms	PASSED (ID ASSIGNED)
4. Shopping Cart Addition	POST /api/cart/items	~40 ms	PASSED (ITEM CACHED)
5. COD Order Placement	POST /api/orders	~85 ms	PASSED (OUTBOX COMMITTED)
6. Kafka Asynchronous Flow	order.events → notification	< 500 ms	PASSED (STOCK RESERVED & ALERTED)
7. Real-Time Admin Sync	GET /api/admin/orders	~38 ms	PASSED (VISIBLE IN DASHBOARD)

Production Cluster Verification Log

```
$ kubectl get pods,svc -n petstore
pod/admin-frontend-7689c89744-4br2b      1/1 Running 0 42m IP: 10.106.128.84
pod/api-gateway-7cb4c8bfbdb-rmc5s      1/1 Running 0 75m IP: 10.106.128.71
pod/customer-frontend-869fd9c68f-m8xc7  1/1 Running 0 42m IP: 10.106.128.85
pod/catalog-service-d889f857-s8qd7      1/1 Running 0 82m IP: 10.106.128.22
pod/inventory-service-769b44bfb4-pw6m8  1/1 Running 0 82m IP: 10.106.128.23
pod/order-service-c6955b554-8ndjw      1/1 Running 1 82m IP: 10.106.128.69
pod/notification-service-7f48794b8c-68xq2 1/1 Running 0 82m IP: 10.106.128.70
pod/user-service-6dc4c5d6fc-gtsrd      1/1 Running 0 82m IP: 10.106.128.21
pod/postgres-0                          1/1 Running 0 85m IP: 10.106.128.20
pod/kafka-0                             1/1 Running 0 88m IP: 10.106.128.17
service/customer-frontend-public LoadBalancer 34.118.227.205 136.65.190.222 80:32324/TCP
service/admin-frontend-public LoadBalancer 34.118.237.135 34.136.230.80 80:30151/TCP
```

Need an Enterprise Cloud or Microservices System Built?

I specialize in turning complex software ideas into production-ready, scalable, and resilient cloud platforms. Whether you need a ground-up microservices architecture, a migration to Kubernetes, or custom backend APIs, I deliver clean, documented, and tested code.

- Spring Boot 3 Microservices
- Kubernetes / GKE / EKS
- Apache Kafka Event Sourcing
- React 19 & TypeScript SPAs
- Docker & CI/CD Pipelines
- PostgreSQL & Database Architecture
- Cloud Run & Serverless
- API Gateways & Security (JWT)